TX

Turn Semantics, 복구, 내구성

한 번의 모델 호출이 아니라 부분 효과와 실패를 일관된 history로 닫는 문제

> **turn은 원자적이지 않다. 그래서 더 엄격한 terminal·pairing·settlement 의미가 필요하다.**

대상 snapshot
v2.1.88 / 2ca5ddabfed5

설명 구조
조건 · 이론 · 구현 · 반례

제3자 복원 소스에 대한 교육용 분석이다. 코드 관찰, 이론적 비유, 개선 제안을 구분하며 누락 모듈을 추정해 채우지 않는다.

turn의 경계는 API response보다 넓다

> 언제 한 turn이 끝났다고 말할 수 있는가?

성립 조건

한 turn 안에 streaming message, tool proposal, 외부 effect, hook, user abort, transcript write가 섞인다.

이론 렌즈

state machine에서 completion은 transition sequence와 terminal state의 의미로 정의된다.

코드의 구체화

query generator가 중간 event를 yield하고 terminal reason을 return하며 QueryEngine/entrypoint가 후처리한다.

깨지는 지점

HTTP stream 종료만 completion으로 보면 tool effect와 durable history가 아직 합류하지 않았을 수 있다.

해석	turn completion은 transport 종료가 아니라 protocol·effect·durability의 settlement 조건이다.

근거와 연결 src/query.ts · src/QueryEngine.ts

transaction 비유는 불변조건에서만 쓴다

어떤 점이 transaction-like하고 어떤 점이 아닌가?

성질	turn에서 대응	차이
일관된 종료	terminal reason과 message pairing	여러 외부 system atomic commit 없음
중간 상태	progress/tool events	소비자에게 의도적으로 노출
abort	generation/tool 취소	이미 실행된 effect rollback 불가
log	transcript/session journal	완전한 WAL/redo protocol 아님
retry	fallback/recovery branch	중복 효과 방지 별도 필요

정확한 표현 turn은 “transaction”이라기보다 transaction-like invariants를 가진 long-running interaction이다.

부분 효과는 rollback보다 compensation을 요구한다

왜 실패 후 이전 상태로 돌아간다고 가정할 수 없는가?

성립 조건

파일 write, process spawn, network call은 이미 외부 세계를 바꿀 수 있다.

이론 렌즈

Saga는 long-lived work를 여러 transaction과 compensating action으로 본다.

코드의 구체화

query는 tool failure/abort를 result와 terminal reason으로 합류시키고 후속 recovery를 선택한다.

깨지는 지점

모든 tool에 inverse가 없고, compensation 자체도 실패할 수 있으므로 database saga 구현과 동일하지 않다.

해석	Saga 비유의 가치는 “error=zero effect” 가정을 버리고 부분 효과 receipt와 후속 정합화를 요구하는 데 있다.

근거와 연결 Garcia-Molina & Salem, Sagas · src/query.ts

State는 변수 묶음이 아니라 transition의 증거다

> query state를 읽을 때 무엇을 찾아야 하는가?

state 종류	결정하는 것	필요한 증거
messages	다음 model world	role/pairing/provenance
tool context	effect capability	permission/abort/session refs
recovery guards	재시도 가능성	attempt/fingerprint/limit
cost/cache	budget transition	token/cache usage
terminal	caller 후처리	reason + final observations

변수가 많다는 사실보다 어떤 transition만 그 값을 바꿀 수 있고, 그 변경이 transcript와 caller에 어떻게 투영되는지가 중요하다.

근거와 연결 src/query.ts

tool_use와 tool_result는 protocol상의 괄호다

왜 orphan message가 단순 표시 오류가 아닌가?

성립 조건

provider와 다음 reasoning은 tool proposal과 관찰을 id로 연결해야 한다.

이론 렌즈

well-formed protocol은 open request가 정확히 대응하는 response 또는 명시적 tombstone으로 닫혀야 한다.

코드의 구체화

model error/fallback path가 missing result를 repair하거나 failed attempt의 projection을 제거한다.

깨지는 지점

orphan tool_use는 다음 request를 invalid하게 만들거나 모델이 effect 결과를 잘못 추론하게 한다.

해석 pairing은 UI 정리 규칙이 아니라 conversation history의 문법 불변조건이다.

근거와 연결 src/query.ts · src/query.ts

삭제 표지는 새 데이터가 아니라 실패한 역사를 보존한다

왜 failed attempt를 그냥 배열에서 지우지 않는가?

fallback 때 이전 attempt의 content가 새 provider/model path에 노출되면 중복 proposal이나 invalid pairing이 생길 수 있다. tombstone은 "이 entity가 없었다"가 아니라 "이 projection에서 무효화되었다"는 lineage signal이다.

provenance 관점	invalidation event를 남기면 observer와 durable history가 실패 전후 entity를 구분할 수 있다.

근거와 연결 src/query.ts · PROV constraints

fallback은 같은 호출을 다른 provider에 다시 보내는 일이 아니다

이미 streaming된 output과 effect가 있다면 무엇이 달라지는가?

성립 조건

첫 attempt가 partial text나 tool proposal을 방출했을 수 있다.

이론 렌즈

retry safety는 operation identity와 effect uncertainty를 보존해야 한다.

코드의 구체화

request/observer projection을 정리하고 cache-related state를 보존하며 recoverable error를 일시 보류한다.

깨지는 지점

첫 attempt의 tool이 실행됐는데 두 번째 attempt가 같은 tool을 다시 제안하면 duplicate effect가 발생한다.

해석 fallback policy는 transport availability와 semantic replay safety를 함께 판정해야 한다.

근거와 연결 src/query.ts · src/services/api/claude.ts

generator는 시간 구조를 값의 타입에 넣는다

Promise>와 무엇이 다른가?

성립 조건

caller는 turn이 끝나기 전 progress·message·tool observation을 소비해야 한다.

이론 렌즈

ECMAScript AsyncGenerator는 suspended state와 request queue를 가지며 next/return/throw completion을 구분한다.

코드의 구체화

query는 중간 AssistantMessage를 yield하고 최종 terminal state를 return한다.

깨지는 지점

consumer가 `.return()`하면 producer cleanup과 외부 effect cancellation이 자동 동치가 아니며, buffer bound도 제공하지 않는다.

해석 generator의 핵심은 반복문 문법이 아니라 producer continuation과 consumer demand 사이의 protocol이다.

근거와 연결 src/query.ts · ECMAScript AsyncGenerator

suspended-yield는 durable checkpoint가 아니다

메모리 continuation과 resume session을 왜 구분해야 하는가?

개념	보존 위치	process 종료 후
generator local frame	runtime memory	소실
yielded event	consumer/UI memory	소비자에 따라 소실
transcript record	session storage	복원 후보
external effect	filesystem/process/remote	독립적으로 지속 가능
resume projection	reader/repair logic	history에서 재구성

오해 교정 generator가 "멈췄다가 이어진다"는 사실은 crash-resume나 durable workflow를 의미하지 않는다.

pull protocol도 무한 producer를 자동으로 통제하지 않는다

UI가 느릴 때 어떤 queue가 커지는가?

성립 조건

API stream, generator queue, React update, terminal output은 서로 다른 처리율을 가진다.

이론 렌즈

WHATWG Streams는 queue와 desiredSize를 통해 backpressure를 명시한다.

코드의 구체화

query yield는 caller가 next를 요청하는 pull 형태지만 내부 provider stream과 progress source는 별도 buffering을 가질 수 있다.

깨지는 지점

UI batch만 조절하고 upstream stream을 멈추지 못하면 memory·latency가 누적된다.

해석	backpressure는 syntax property가 아니라 전체 producer-queue-consumer chain의 계약이다.

근거와 연결 WHATWG Streams · src/services/api/claude.ts

취소는 신호이지 과거를 지우는 명령이 아니다

AbortSignal이 보장하는 것과 보장하지 않는 것은 무엇인가?

성립 조건

여러 await와 tool adapter가 같은 turn의 cancellation을 관찰한다.

이론 렌즈

AbortController는 signal을 한 번 발생시키고 consumer가 협력적으로 반응하게 한다.

코드의 구체화

query와 tool context가 abort state를 검사하고 terminal/cleanup path로 이동한다.

깨지는 지점

signal을 무시하는 child process, 이미 완료된 write, finally 누락은 그대로 남는다.

해석 취소 correctness는 signal 전달, effect-specific interruption, cleanup, durable reconciliation 네 층으로 나뉜다.

근거와 연결 src/query.ts · Node AbortController

종료 reason은 boolean보다 풍부한 caller 계약이다

> completed=false 하나로 무엇을 잃는가?

reason family	보존하는 의미	caller 선택
completed	정상 settlement	result/usage 후처리
blocking/context limit	budget policy 중단	compact/새 turn
model/provider error	transport/runtime 실패	fallback/표시
aborted tools	부분 효과 가능	reconcile/interrupt 표시
hook stopped	정책에 의한 중단	이유와 state 보존
max turns	운영 한도	resume 가능한 종료

복원본의 transition type 선언 일부는 누락되어 있으므로 variant 목록은 return 지점과 caller 처리에서 재구성한 관찰이다.

근거와 연결 src/query.ts · src/QueryEngine.ts

transcript를 일찍 쓰는 것은 latency와 recoverability의 tradeoff다

왜 query 전 user message를 기록하는가?

성립 조건

process가 generation 중 kill되어도 사용자 입력은 resume에 남아야 한다.

이론 렌즈

write-ahead 모양은 effect 이전 intent를 durable record로 남긴다.

코드의 구체화

QueryEngine은 loop 시작 전 user message를 transcript에 기록하고 명시적으로 latency tradeoff를 주석화한다.

깨지는 지점

기록 성공 뒤 API 호출 전 crash하면 durable history에는 unanswered input이 남는다.

해석 early persistence는 data loss를 줄이는 대신 incomplete-turn repair 의무를 만든다.

근거와 연결 src/QueryEngine.ts

sessionStorage는 저장 함수가 아니라 reconciliation system이다

왜 persistence 파일이 커지는가?

queue, flush, transcript append, resume, repair, compaction projection, compatibility가 한 곳에 모이면 “쓰기”보다 “역사 해석” 책임이 커진다. 크기는 분해 근거가 아니라 서로 다른 consistency model이 공존한다는 탐색 신호다.

관심사	불변조건	변화 이유
append	순서·identity	runtime event
flush	durability boundary	I/O/platform
reader	old format 해석	compatibility
repair	orphan/incomplete 보정	failure semantics
projection	consumer view	SDK/resume/UI

근거와 연결 src/utils/sessionStorage.ts · src/utils/sessionStorage.ts

transcript는 event log와 닮았지만 곧바로 event sourcing은 아니다

어떤 조건이 더 필요할까?

필요 조건	현재 관찰	unknown
append-only identity	message records/ids	모든 mutation 규칙
versioned event schema	metadata/compat code	공식 schema owner
deterministic projection	resume/repair logic	완전 replay corpus
effect correlation	tool ids/request refs	외부 effect receipt 완전성
snapshot/compaction	compact projection	lossless rebuild 가능 범위

정확한 표현 event-log 성격과 projection은 관찰되지만 완전한 event-sourced architecture라고 단정하지 않는다.

turn 사이 사용자 입력은 동시성 문제다

현재 turn이 끝나기 전 새 prompt는 어디에 속하는가?

성립 조건

interactive user는 tool 실행/stream 중 새 command를 보낼 수 있다.

이론 렌즈

linearization point가 없으면 old turn과 new intent의 순서가 모호해진다.

코드의 구체화

queued commands와 context modifiers가 current/next state에 선택적으로 합류한다.

깨지는 지점

cancel과 resubmit이 엇갈리면 stale finally가 새 prompt state를 복구하거나 지울 수 있다.

해석	input queue는 UX buffer가 아니라 conversation order를 정의하는 concurrency boundary다.

근거와 연결 src/query.ts · src/screens/REPL.tsx

recovery는 exception type이 아니라 상태 전이 표가 필요하다

> 어떤 조건이 같아야 retry가 허용되는가?

관찰	effect certainty	허용 후보
request 전 실패	none	bounded retry
text 일부 수신	none	projection reset/fallback
tool proposal 수신, 미실행	none if proven	repair + retry
tool 실행 시작	unknown/partial	read-back/reconcile
write confirmed, result lost	effect yes	synthesize observation; no blind retry
same fingerprint 반복	unchanged	stop/escalate

복구 의미를 trace로 고정한 뒤 구조를 바꾼다

> 어떤 trace가 동등성 oracle인가?

01	input	messages, tool pool, policy, abort timing, provider attempt
02	events	yielded message/progress/tool ids 순서
03	effects	filesystem/process diff와 receipt
04	terminal	reason, usage, pending state
05	durability	journal records와 resume projection
06	comparison	허용된 adapter 차이와 금지된 semantic drift

turn을 닫는다는 것은 역사를 합의하는 일이다

이 권에서 남겨야 할 한 문장은 무엇인가?

> **stream이 끝났다는 사실보다, proposal·effect·observation·history가 같은 사건을 가리키는지가 중요하다.**

async generator는 이 합의 과정을 점진적으로 전달한다. 하지만 abort, duplicate effect, transcript durability, resume repair의 의미는 별도 owner와 contract가 닫아야 한다.